_Task1_Iris_Classification.ipynb?

File Edit View Run Kernel Settings Help

Kernel Unknown Trusted

Code

Open in... Python [conda env:base] *

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
sns.set_theme(style="whitegrid")
df = pd.read_csv(r'C:\Users\saina\Downloads\archive\Iris.csv')
print("--- Dataframe Head (First 5 Rows) ---")
print(df.head())
print("\n--- DataFrame Info ---")
print(df.info())
print("\n--- Summary Statistics ---")
print(df.describe())
print("\n--- Missing Values Check ---")
print(df.isnull().sum())
if 'id' in df.columns:
    df = df.drop(columns=['id'])
elif 'Id' in df.columns:
    df = df.drop(columns=['Id'])
s_col = 'sepal_length'
y_col = 'sepal_width'
p_len = 'petal_length'
p_wid = 'petal_width'
target_col = 'species'
plt.figure(figsize=(8, 5))
sns.scatterplot(data=df, x=x_col, y=y_col, hue=target_col, palette='Set1')
plt.title('Plot 1: Sepal Length vs Sepal Width')
plt.tight_layout()
plt.show()
plt.figure(figsize=(8, 5))
sns.boxplot(
    data=df,
    x=target_col,
    y=p_len,
    hue=target_col,
    palette='Set2',
    legend=False
)
plt.figure(figsize=(8, 5))
sns.histplot(data=df, x=p_wid, hue=target_col, kde=True, multiple='stack', palette='muted')
plt.title('Plot 3: Distribution of Petal Width')
plt.tight_layout()
plt.show()
sns.pairplot(df, hue=target_col, palette='Dark2')
plt.suptitle('Plot 4: Pairplot of Iris Features', y=1.02)
plt.show()
X = df.drop(columns=[target_col])
y = df[target_col]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
models = {
    "k-Nearest Neighbors": KNeighborsClassifier(n_neighbors=3),
    "Logistic Regression": LogisticRegression(max_iter=200),
    "Decision Tree": DecisionTreeClassifier(random_state=42)
}
best_accuracy = 0
best_model_name = ""
best_model_instance = None
print("--- MODEL EVALUATION & COMPARISON ---")
for name, model in models.items():
    X_tr = X_train_scaled if name != "Decision Tree" else X_train
    X_te = X_test_scaled if name != "Decision Tree" else X_test
    model.fit(X_tr, y_train)
    y_pred = model.predict(X_te)
    acc = accuracy_score(y_test, y_pred)
    print(f"{name} Accuracy: {acc:.4f}")
    if acc > best_accuracy:
        best_accuracy = acc
        best_model_name = name
        best_model_instance = model
print(f"\n>>> Selected Best Model: {best_model_name} ({best_accuracy*100:.2f}% Accuracy)")
X_te_final = X_test_scaled if best_model_name != "Decision Tree" else X_test
final_preds = best_model_instance.predict(X_te_final)
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, final_preds))
print("\nClassification Report:")
print(classification_report(y_test, final_preds))
joblib.dump(best_model_instance, 'best_iris_model.pkl')
print("Best model saved successfully!")
```

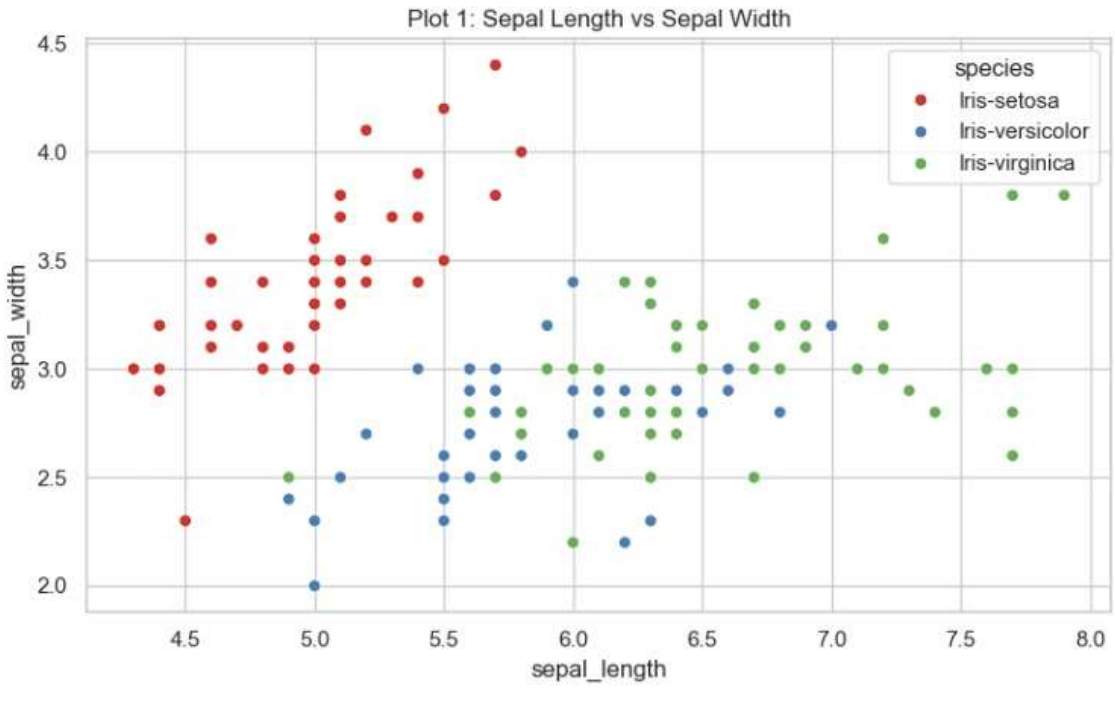
[ ]:

```
    if acc > best_accuracy:
        best_accuracy = acc
        best_model_name = name
        best_model_instance = model
print(f"\n>>> Selected Best Model: {best_model_name} ({best_accuracy*100:.2f}% Accuracy)")
X_te_final = X_test_scaled if best_model_name != "Decision Tree" else X_test
final_preds = best_model_instance.predict(X_te_final)
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, final_preds))
print("\nClassification Report:")
print(classification_report(y_test, final_preds))
joblib.dump(best_model_instance, 'best_iris_model.pkl')
print("Best model saved successfully!")
```

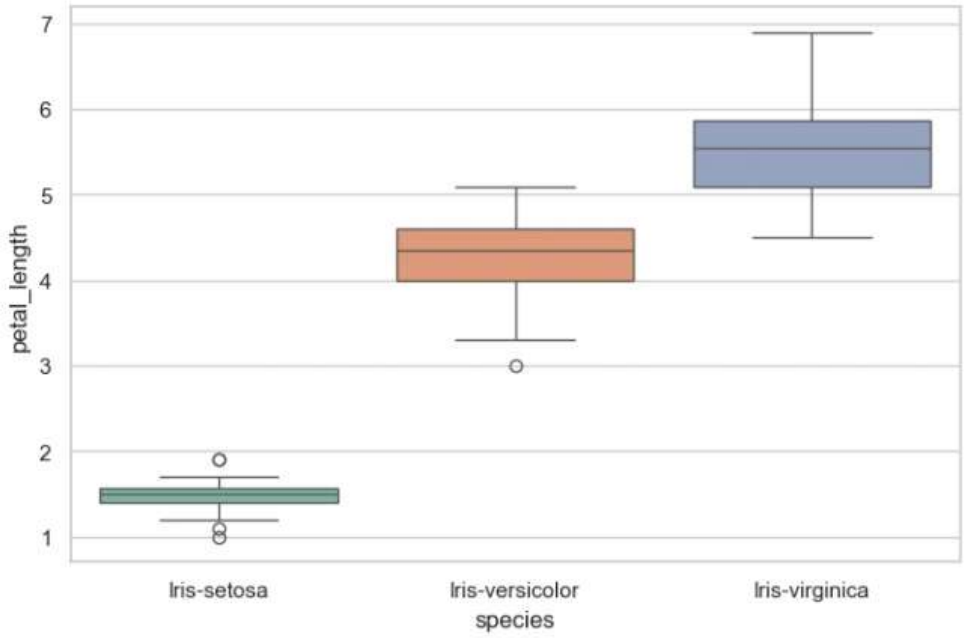
```
--- DataFrame Head (First 5 Rows) ---
   sepal_length  sepal_width  petal_length  petal_width      species
0           5.1          3.5           1.4          0.2  Iris-setosa
1           4.9          3.0           1.4          0.2  Iris-setosa
2           4.7          3.2           1.3          0.2  Iris-setosa
3           4.6          3.1           1.5          0.2  Iris-setosa
4           5.0          3.6           1.4          0.2  Iris-setosa

--- DataFrame Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None

--- Summary Statistics ---
       sepal_length  sepal_width  petal_length  petal_width
count    150.000000   150.000000    150.000000   150.000000
mean       5.843333     3.054000      3.758667     1.198667
std        0.828066     0.433594      1.764420     0.763161
min        4.300000     2.000000      1.000000     0.100000
25%        5.100000     2.800000      1.600000     0.300000
50%        5.800000     3.000000      4.350000     1.300000
75%        6.400000     3.300000      5.100000     1.800000
max        7.900000     4.400000      6.900000     2.500000
```

```
    if acc > best_accuracy:
        best_accuracy = acc
        best_model_name = name
        best_model_instance = model
print(f"\n>>> Selected Best Model: {best_model_name} ({best_accuracy*100:.2f}% Accuracy)")
X_te_final = X_test_scaled if best_model_name != "Decision Tree" else X_test
final_preds = best_model_instance.predict(X_te_final)
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, final_preds))
print("\nClassification Report:")
print(classification_report(y_test, final_preds))
joblib.dump(best_model_instance, 'best_iris_model.pkl')
print("Best model saved successfully!")
```

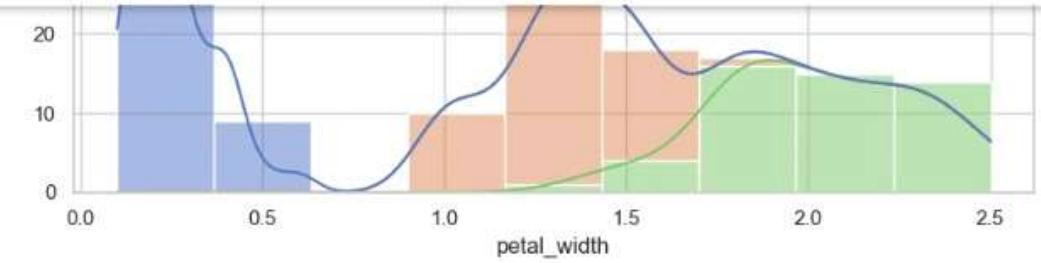
```
--- Missing Values Check ---
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
dtype: int64
```



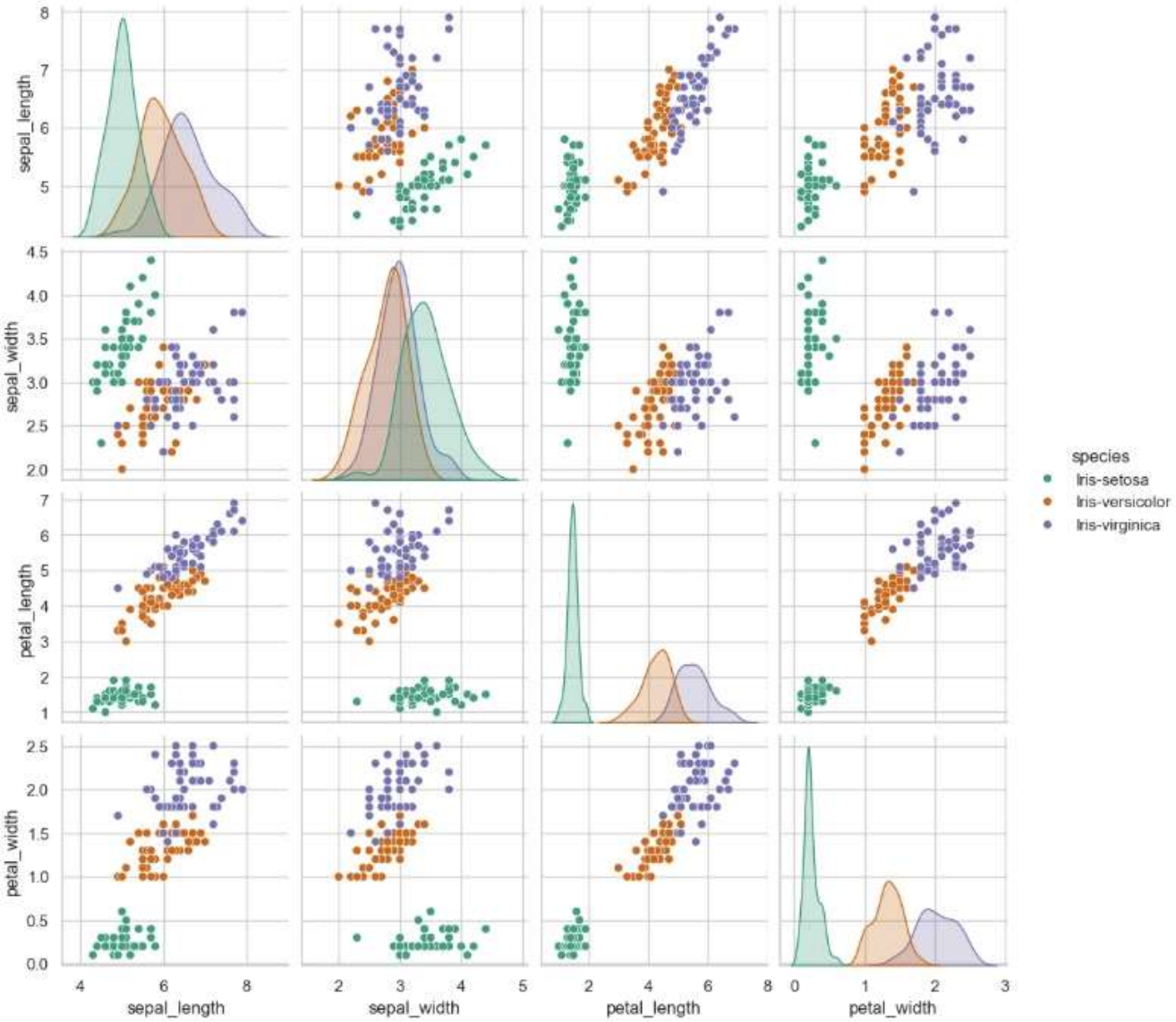
```
        best_model_instance = model
print(f"\n>>> Selected Best Model: {best_model_name} ({best_accuracy*100:.2f}% Accuracy)")
X_te_final = X_test_scaled if best_model_name != "Decision Tree" else X_test
final_preds = best_model_instance.predict(X_te_final)
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, final_preds))
print("\nClassification Report:")
print(classification_report(y_test, final_preds))
joblib.dump(best_model_instance, 'best_iris_model.pkl')
print("Best model saved successfully!")
```



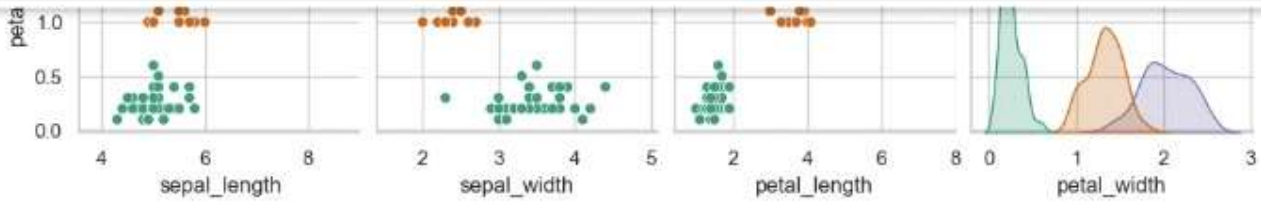

Plot 3: Distribution of Petal Width
species
Iris-setosa
Iris-versicolor
Iris-virginica
Count
0
10
20
30
40
50
60
0.0
0.5
1.0
1.5
2.0
2.5
petal_width



Plot 4: Pairplot of Iris Features



```
    "Logistic Regression": LogisticRegression(max_iter=200),
    "Decision Tree": DecisionTreeClassifier(random_state=42)
}
best_accuracy = 0
best_model_name = ""
best_model_instance = None
print("--- MODEL EVALUATION & COMPARISON ---")
for name, model in models.items():
    X_tr = X_train_scaled if name != "Decision Tree" else X_train
    X_te = X_test_scaled if name != "Decision Tree" else X_test
    model.fit(X_tr, y_train)
    y_pred = model.predict(X_te)
    acc = accuracy_score(y_test, y_pred)
    print(f"{name} Accuracy: {acc:.4f}")
    if acc > best_accuracy:
        best_accuracy = acc
        best_model_name = name
        best_model_instance = model
print(f"\n>>> Selected Best Model: {best_model_name} ({best_accuracy*100:.2f}% Accuracy)")
X_te_final = X_test_scaled if best_model_name != "Decision Tree" else X_test
final_preds = best_model_instance.predict(X_te_final)
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, final_preds))
print("\nClassification Report:")
print(classification_report(y_test, final_preds))
joblib.dump(best_model_instance, 'best_iris_model.pkl')
print("Best model saved successfully!")
```




```
--- MODEL EVALUATION & COMPARISON ---
k-Nearest Neighbors Accuracy: 0.9333
Logistic Regression Accuracy: 0.9333
Decision Tree Accuracy: 0.9333

>>> Selected Best Model: k-Nearest Neighbors (93.33% Accuracy)

Confusion Matrix:
[[10  0  0]
 [ 0 10  0]
 [ 0  2  8]]

Classification Report:
                 precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        10
Iris-versicolor       0.83      1.00      0.91        10
 Iris-virginica       1.00      0.80      0.89        10

       accuracy                           0.93        30
      macro avg       0.94      0.93      0.93        30
   weighted avg       0.94      0.93      0.93        30

Best model saved successfully!
```